Chapter 16: Strings and Sequences

EECS 281 notes, condensed. Covers 16.1 to 16.7.

Legend

- **Red box** = section marked ❃ in the textbook. Treated here as optional / lower priority (the chapter does not define ❃, so check your syllabus). These are:
  - 16.2.1 String Implementations and Short String Optimization
  - 16.2.3 `std::string_view`
  - 16.2.4 Additional String Formatting Methods (`std::format`, `std::print`)
  - 16.7 Knuth-Morris-Pratt (KMP) String Searching, including 16.7.1–16.7.3

  Everything else (16.1, 16.2.2, 16.3–16.6) is unmarked core material.
- ⋆ = core, high-yield material worth memorizing.
- Boxes: **blue** = definition, **green** = worked example, **yellow** = remark / watch out.

16.1 C Strings

C string: a `char` array terminated by the null / **sentinel** character `'\0'` (ASCII 0). The sentinel is how code finds the end, so every C string needs one. A C string variable decays to a pointer to its first character, so `==` compares *addresses*, not contents. Use `<cstring>` functions instead.

Declaring "EECS"

```
const char* cstr = "EECS";              // read-only literal, sentinel added for you
char cstr[] = "EECS";                   // sentinel added for you (size 5)
char cstr[5] = {'E','E','C','S','\0'};  // explicit size: add '\0' yourself
char* cstr = new char[5];               // heap: add '\0' yourself
```

ASCII

- Each `char` is stored as a number. In memory, "EECS" is `69 69 67 83 0`.
- Key values: `'\0'` = 0, `'\t'` = 9, `'\n'` = 10, `'\r'` = 13, space = 32, `'0'` = 48, `'A'` = 65, `'a'` = 97.
- ⋆ **Digits < uppercase < lowercase** (48–57, 65–90, 97–122).
- Chars are numbers, so you can compare and loop over them: `for (char c = 'a'; c <= 'z'; ++c)`.

<cstring> functions

Function	What it does	Gotcha
`strlen(s)`	Length, not counting `'\0'`	⋆ **Θ(n)**: walks to the sentinel
`strcpy(dest, src)`	Copies `src` including `'\0'`	`dest` must be big enough; no overlap
`strncpy(dest, src, num)`	Copies first `num` chars; pads with zeros if `src` is shorter	⋆ If `src` is longer, **no sentinel** is added
`strcat(dest, src)`	Appends `src`, overwriting `dest`'s `'\0'`, adds a new one	`dest` must have room
`strncat(dest, src, num)`	Appends first `num` chars **plus** a sentinel	Always terminates
`strcmp(a, b)`	0 if equal, negative if a < b, positive if a > b (by ASCII)	Compare result to 0, not to ±1
`strncmp(a, b, num)`	Same, but only the first `num` chars	`strncmp("apple","application",4) == 0`

Remark: `std::cout << char_ptr` prints until the first `'\0'`. A `char*` with no sentinel gives **undefined behavior**.

16.2 C++ Strings

- `std::string` (in `<string>`) is a class that owns and manages its character array, like a `std::vector<char>`.
- ⋆ It stores its length, so `.size()` is **Θ(1)**, vs `strlen()` Θ(n).

❃ OPTIONAL · 16.2.1

16.2.1 Short String Optimization

- A typical (libstdc++) `std::string` is 32 bytes on the stack: data pointer (8) + length (8) + a 16-byte union.

```
class string {
  char* data;
  size_t length;
  union {
    size_t capacity;       // used when data lives on the heap
    char local_buffer[16]; // used for short strings (SSO)
  };
};
```

- Long strings: data on the heap; the union holds the capacity.
- ⋆ **SSO:** strings of length **≤ 15** are stored in the 16-byte buffer on the stack, avoiding a heap allocation. Length 16 does not fit, since `'\0'` needs a 17th byte.

16.2.2 Common operations

Strings support =, <, >, ==, and random access iterators. Each example applies the operation to a `std::string` holding the value shown.

Operation	Behavior	Example
`.size()` / `.length()`	Number of chars, not counting `''`. The two are identical. Θ(1)	`"EECS281"` → 7
`.clear()`	Removes all chars; size becomes 0	`"EECS281"` → `""`
`.empty()`	Returns `true` if size is 0 (does not modify)	`""` → `true`
`[idx]`	Reference to the char at `idx`; no bounds check	`"EECS281"[2]` → `'C'`
`.front()`	Reference to the first char. **UB if empty**	`"EECS281"` → `'E'`
`.back()`	Reference to the last char. **UB if empty**	`"EECS281"` → `'1'`
`+=`	Appends a string, C string, or char	`"hello " += "world"` → `"hello world"`
`.push_back(c)`	Appends one char to the end	`"EECS28".push_back('1')` → `"EECS281"`
`.pop_back()`	Removes the last char. **UB if empty**	`"EECS281"` → `"EECS28"`
`.insert(pos, str)`	Inserts `str` before index `pos`	`"pot".insert(1, "arr")` → `"parrot"`
`.erase(pos, len)`	Erases `len` chars starting at `pos` (defaults: pos 0, len npos = to the end)	`"carrot".erase(2, 3)` → `"cat"` `"carrot".erase(3)` → `"car"`
`.erase(it)`	Erases the one char at iterator `it`; returns iterator to the next char	`"carrot".erase(begin() + 1)` → `"crrot"`
`.erase(first, last)`	Erases the iterator range [first, last)	`"carrot".erase(begin(), begin() + 3)` → `"rot"`
`.replace(pos, len, str)`	Replaces `len` chars starting at `pos` with `str`	`"eecs 370 is fun".replace(5, 3, "281")` → `"eecs 281 is fun"`
`.c_str()`	Returns a `const char*` to the null-terminated data	`"eecs 281"` → `const char*` to `"eecs 281"`
`.substr(pos, len)`	Returns a new string of `len` chars from `pos` (len defaults to npos)	`"pineapple".substr(4, 3)` → `"app"`
`.find(str, pos)`	Index where the **whole** `str` first appears, searching from `pos`; npos if none	`"pineapple".find("apple")` → 4
`.find_first_of(chars, pos)`	Index of the first char that is **any** char in `chars`, searching forward from `pos`	`"pineapple".find_first_of("orange")` → 2 (`'n'`)
`.find_last_of(chars, pos)`	Index of the last char that is any char in `chars`, only looking at indices ≤ `pos`	`"pineapple".find_last_of("apple")` → 8 (`'e'`)
`.find_first_not_of(chars, pos)`	Index of the first char that is **not** in `chars`, searching forward from `pos`	`"pineapple".find_first_not_of("pin")` → 3 (`'e'`)
`.find_last_not_of(chars, pos)`	Index of the last char that is **not** in `chars`, only looking at indices ≤ `pos`	`"pineapple".find_last_not_of("ple")` → 4 (`'a'`)
`a + b` (non-member)	Returns a new string: `a` followed by `b`	`std::string("pine") + "apple"` → `"pineapple"`

`std::string::npos`: a constant holding the largest `size_t` value. It has two uses:

1. **As an argument, it means "go to the end."** `len` defaults to npos, so `s.substr(4)` = `s.substr(4, npos)` = everything from index 4 on, and `s.erase(3)` erases from index 3 to the end.
2. **As a return value from any `find`, it means "not found":** `if (s.find("orange") == std::string::npos) { /* not in s */ }`

Store the result in a `size_t`, not an `int`. npos does not fit in an `int`, so comparing it to `-1` or a negative value is a classic bug.

Watch out, two textbook errors in `find_last_of`:

1. **Wrong example answer.** It says `"pineapple".find_last_of("pie", 4)` returns 5. The `4` means only indices 0–4 (`p i n e a`) are considered. The last of those that is a `p`, `i`, or `e` is the `'e'` at index **3**. Index 5 is a `'p'`, but it is past position 4, so it is never searched.
2. **Wrong default.** The description says `pos` defaults to 0, but the signature shows the real default, **npos**, which searches the whole string. That is why `find_last_of("apple")` returns 8. With a default of 0 it would only ever check index 0.

❄ OPTIONAL · 16.2.3

16.2.3 std::string_view

std::string_view (C++17, `<string_view>`): a **non-owning, read-only view** of someone else's characters. It is just a pointer + length (16 bytes).

- ★ Use it to avoid copies. `substr()` on a view returns another view with **no allocation**; on a `std::string` it allocates a new string.
- Example: parsing `Timestamp:...;Uniqname:...;Assignment:...` lines. The `std::string` version allocates ~7 strings per line; the `string_view` version allocates only once (the `getline`).
- Takes a string literal without allocating, unlike `const std::string&`, which builds a temporary string (on the heap if too long for SSO).

```
std::string str = "xxxEECS281xx";
std::string_view v = str;
v.remove_prefix(3);                       // move start forward
v.remove_suffix(2);                       // move end backward
std::cout << v;                           // "EECS281"
std::string_view v2{str.c_str() + 7, 3};  // "281" (pointer + length)
```

Remark: **pass `string_view` by value**, not by const reference. It is small and trivially copyable, and by-value avoids an extra indirection.
Pitfalls: a view must not **outlive** the string it views (dangling). It is not guaranteed to be **null-terminated**, so never pass `.data()` to C string functions. If the function needs a `std::string` or a `'\0'` anyway, prefer `const std::string&`.

❄ OPTIONAL · 16.2.4

16.2.4 Formatting

- C++20 `std::format` (`<format>`): `{}` placeholders, or `{0}`, `{1}` to choose argument order. C++23 adds `std::print()` / `std::println()`.

```
std::format("My favorite class is {} {}!", "EECS", "281");
std::format("{1} is the best class in {0}", "EECS", "281");
```

16.3 Lexicographical String Comparisons

- `std::string` comparison operators use dictionary order **by ASCII value**:
  `"" < "Zebra" < "a" < "ab" < "b" < "bb" < "bc" < "bc0"`
- ⋆ Digits compare as characters, not numbers: `"ab10" < "ab9"` because `'1' < '9'`.

`std::lexicographical_compare(first1, last1, first2, last2, comp)` (`<algorithm>`): returns `true` if range 1 is **strictly less** than range 2. `comp` is optional (default `operator<`). Works across container types (e.g. a `std::string` vs a `std::deque<char>`).

⋆ Rules

1. Compare element by element; the **first mismatch** decides.
2. If one range is a **prefix** of the other, the shorter one is less (`"app" < "apple"`).
3. Same length and same elements → equal, so it returns **false** (strict less).
4. An empty range is less than any non-empty range; two empty ranges are equal.

Example: `"apple"` vs `"Banana"` → **false**, since `'B'` (66) < `'a'` (97). Watch the types too: `'3'` (51) is not the integer `3`.

Reference implementation

```
for (; first1 != last1 && first2 != last2; ++first1, ++first2) {
  if (*first1 < *first2) return true;
  if (*first2 < *first1) return false;
}
return first1 == last1 && first2 != last2;  // range 1 ran out first
```

Remark: only `<` and `==` are needed for all six comparisons: `a != b = !(a == b)`, `a > b = b < a`, `a <= b = !(b < a)`, `a >= b = !(a < b)`.

16.4 STL Sequence Operations

std::equal(first1, last1, first2, pred)

- Returns `true` if [first1, last1) matches the same number of elements starting at `first2`. `pred` is optional (default ==). Works on C strings, where == does not.
- ⋆ **The first range sets the length.** Only that many elements of the second range are checked.

Example:

```
equal("app".begin(),   "app".end(),   "apple".begin())  // true: only checks "app"
equal("apple".begin(), "apple".end(), "app".begin())    // false: reads past "app"
```

```
bool is_prefix(const std::string& prefix, const std::string& s) {
  return std::equal(prefix.begin(), prefix.end(), s.begin());
}
bool is_palindrome(const std::string& s) {   // front half vs reversed back half
  return std::equal(s.begin(), s.begin() + s.size() / 2, s.rbegin());
}
```

std::unique(first, last, pred)

- Keeps only the **first** of each run of **consecutive** equal elements, moving kept elements to the front.
- ⋆ Returns an iterator **one past the last kept element**. It cannot shrink the container, so you must `erase` the tail yourself.
- ⋆ It does **not sort**. On unsorted data, non-adjacent duplicates survive. Sort first to remove all duplicates.

```
std::sort(vec.begin(), vec.end());
auto it = std::unique(vec.begin(), vec.end());   // first element to erase
vec.erase(it, vec.end());
```

Example: `{5,5,5,3,4,4,4,3,3,3,6,7,7,6,7}` → sort → unique → erase → `{3,4,5,6,7}`.
On a string: `"This pizza is really good!!!"` → removes one `z`, one `l`, one `o`, two `!` (5 chars).

16.5 Brute Force String Searching

Needle in a haystack: find every position where the needle (length **n**) occurs in the haystack (length **h**).
Sliding window: compare the needle against every length-n window, stopping each window at its first mismatch.

```
for (size_t i = 0; i <= h - n; ++i) {
  size_t j = 0;
  while (j < n && haystack[i + j] == needle[j]) ++j;
  if (j == n) matches.push_back(i);
}
```

- ⋆ **Average Θ(h):** most windows fail within the first character or two, so each check is Θ(1).
- ⋆ **Worst Θ(nh):** every window matches until its last character, e.g. haystack `aaaa…ab`, needle `aaaaaab`.
- Fast in practice, since the worst case is rare.

Watch out: `h - n` is a `size_t`. If the needle is longer than the haystack it underflows. Guard with `if (n > h) return {};`.

Best Time	Average Time	Worst Time	Auxiliary Space
Θ(h)	Θ(h)	Θ(nh)	Θ(1)

16.6 Rabin-Karp String Searching

- ⋆ Idea: compare integer **fingerprints** (a **rolling hash**) of each window instead of the strings. An integer compare is Θ(1).

> **Fingerprints only prove difference.** Different fingerprints → strings definitely differ. Equal fingerprints → strings *might* be equal (a **collision** / false positive), so verify character by character.

Choosing a fingerprint

- **Bad:** sum of ASCII values ignores position. `"cat"`, `"act"`, `"kid"` all = 312, causing many collisions.
- ⋆ **Rabin fingerprint:** multiply the running value by base **k**, then add the next character. Position now matters:

$$FP(c_1c_2...c_n) = k^{n-1}c_1 + k^{n-2}c_2 + ... + k^1c_{n-1} + k^0c_n$$

> **Example (k = 10):**
>
> ```
> "cat":  99 → 10×99 + 97 = 1087 → 10×1087 + 116 = 10986
> "act":  97 → 10×97 + 99 = 1069 → 10×1069 + 116 = 10806   (now different)
> ```

- Pick k as a power of 2 (multiplying is a bit shift).
- Take every result **mod a large prime** to prevent overflow and spread values uniformly. If a value goes negative, add the prime back.

⋆ Rolling update in Θ(1)

> **Example:** slide from window `abc` (10779) to `bcd`, with k = 10, n = 3.
>
> ```
> 1. Remove leaving char:  10779 − 10²×97 = 1079     ("bc")
> 2. Shift by k:           1079 × 10      = 10790
> 3. Add entering char:    10790 + 100    = 10890    ("bcd")
> ```
>
> Only two fingerprints are built from scratch (needle + first window), each Θ(n). Every later window is Θ(1).

Algorithm

1. Compute FP(needle) and FP(first window), both Θ(n). Precompute `left_multiplier` = k^{n-1} mod prime.
2. For each window: if fingerprints match, brute-force verify, then record the match if it is real.
3. Slide: `fp = (k × (fp − haystack[i] × left_multiplier) + haystack[i + n]) % prime`, fixing negatives.

```
for (size_t i = 0; i <= h - n; ++i) {
  if (fp_needle == fp_hay &&
      std::equal(needle.begin(), needle.end(), haystack.begin() + i))
    matches.push_back(i);
  int64_t no_left = fp_hay - haystack[i] * left_multiplier;
  fp_hay = (base * no_left + haystack[i + n]) % prime;
  if (fp_hay < 0) fp_hay += prime;
}
```

> **Watch out:** the textbook's constants (base 128, prime ≈ 3.8×10^{17}) overflow `int64_t`: `128 × fp` can reach ~4.8×10^{19}. `std::pow` also returns a `double`, which loses precision for large n. In real code, keep prime < 2^{55} / base (or use `__int128`), and build k^{n-1} with a modular loop.

Complexity

- ⋆ **Average Θ(n + h):** Θ(n) to build fingerprints, Θ(1) per window across h windows, plus Θ(n) to verify each real match.
- ⋆ **Worst Θ(nh):** a terrible hash collides at every window, forcing a Θ(n) verify each time. Extremely unlikely with a good hash.
- Also good for **fixed-length repeated-substring** problems: store each window's fingerprint, Θ(n) average.

Best Time	Average Time	Worst Time	Auxiliary Space
Θ(n + h)	Θ(n + h)	Θ(nh)	Θ(1)

❄ OPTIONAL · 16.7

16.7 Knuth-Morris-Pratt (KMP) String Searching

- ⋆ Idea: after a mismatch, reuse what already matched. The haystack pointer **never moves backward**, giving a worst case of Θ(n + h).
- **Prefix** = a substring starting at index 0. **Suffix** = a substring ending at the last index.

⋆ Prefix table (π / LPS table)

> **prefix[i]** = length of the longest **proper** prefix of `needle[0..i]` that is also a suffix of it. Proper means the whole string is excluded, so `prefix[0] = 0` always. Overlap between the prefix and suffix is allowed (`"aabaa"` → 2).

> **Examples:**

needle	a	b	d	c	d	a	b	c	b
prefix	0	0	0	0	0	1	2	0	0

needle	a	a	b	c	b	a	a	a	b	c	a	b
prefix	0	1	0	0	0	1	2	2	3	4	1	0

needle	a	a	b	a	a	b	a	a	a
prefix	0	1	0	1	2	3	4	5	2

⋆ Search rules

1. Compare haystack and needle characters while they match.
2. **Mismatch at needle index j > 0:** set `j = prefix[j − 1]` and compare the **same** haystack char again. The first `prefix[j − 1]` chars are already known to match.
3. **Mismatch at j = 0:** advance the haystack by one.
4. **Full match:** record `curr − n + 1`, then continue with `j = prefix[n − 1]`.

Code

```
std::vector<size_t> compute_prefix_array(const std::string& needle) {
  std::vector<size_t> pre(needle.length());
  size_t j = 0;
  for (size_t i = 1; i < needle.length(); ++i) {
    while (j > 0 && needle[j] != needle[i]) j = pre[j - 1];  // fall back
    if (needle[j] == needle[i]) ++j;                         // extend
    pre[i] = j;
  }
  return pre;
}

std::vector<size_t> kmp(const std::string& needle, const std::string& haystack) {
  std::vector<size_t> matches, pre = compute_prefix_array(needle);
  size_t idx = 0;
  for (size_t curr = 0; curr < haystack.length(); ++curr) {
    while (idx > 0 && needle[idx] != haystack[curr]) idx = pre[idx - 1];
    if (needle[idx] == haystack[curr]) ++idx;
    if (idx == needle.length()) {
      matches.push_back(curr - needle.length() + 1);
      idx = pre[idx - 1];   // missing in the textbook
    }
  }
  return matches;
}
```

- Building the table is the same algorithm as the search, matching the needle against itself.

> **Watch out:** the textbook's search omits the reset after a match. The next comparison then reads `needle[n]` (the `'\0'`), which never matches a real char, so it falls back on its own. The explicit reset is the correct form.

⋆ Complexity

- **Time Θ(n + h):** Θ(n) for the table and Θ(h) for the search, in the worst case too.
- **Why linear (amortized):** `j` rises by at most 1 per outer iteration, and each `while` step strictly decreases it (since `pre[j − 1] < j`). It cannot drop below 0, so total fall-backs ≤ total increments ≤ loop length.
- **Space Θ(n)** for the prefix table.

> **Remark:** the prefix table alone solves **longest prefix that is also a suffix** of a string: answer = `str.substr(0, pre.back())`. That is linear in the worst case, vs Θ(n^2) worst case for a rolling-hash approach.

Best Time	Average Time	Worst Time	Auxiliary Space
Θ(n + h)	Θ(n + h)	Θ(n + h)	Θ(n)

Summary of Complexities

n = needle length, h = haystack length.

String search method	Average Time	Worst Time	Auxiliary Space
Brute force (sliding window)	Θ(h)	Θ(nh)	Θ(1)
Rabin-Karp	Θ(n + h)	Θ(nh)	Θ(1)
KMP (❃ optional)	Θ(n + h)	Θ(n + h)	Θ(n)

Related operation	Time
`strlen()` on a C string	Θ(n)
`std::string::size()`	Θ(1)
Compare two strings of length m	Θ(m) worst
`std::sort` on n strings of length m	Θ(mn log n) worst
Binary search a sorted array of n strings of length m	Θ(m log n)